SIMPLE NEURAL NETWORK FOR MNIST CLASSIFICATION

Abstract

In this paper, we present a simple neural network architecture for classifying handwritten digits from the MN

1. Introduction

The MNIST dataset is a widely used benchmark for evaluating machine learning algorithms, particularly in

Despite the availability of more complex architectures such as convolutional neural networks (CNNs), we s

2. Method

Our approach uses a simple feedforward neural network with two hidden layers. The architecture is as follo

- Input layer: 784 neurons (28x28 flattened images)
- First hidden layer: 128 neurons with ReLU activation
- Second hidden layer: 64 neurons with ReLU activation
- Output layer: 10 neurons with softmax activation

We use the cross-entropy loss function and the Adam optimizer with a learning rate of 0.001. We train the

Algorithm 1: Training the Neural Network

1. Load the MNIST dataset
2. Normalize the pixel values to [0, 1]
3. Flatten the 28x28 images to 784-dimensional vectors
4. Initialize the neural network with the architecture described above
5. For each epoch:

a. For each batch of training data:

i. Forward pass: compute predictions

ii. Compute loss

iii. Backward pass: compute gradients

iv. Update weights using Adam optimizer

b. Evaluate on validation set

6. Evaluate on test set

3. Experimental Setup

We use the standard MNIST dataset split: 60,000 training images and 10,000 test images. We further split

We implement our approach using PyTorch. All experiments are run on a single CPU core. We report the a

4. Results

Our simple neural network achieves 97.5% accuracy on the MNIST test set. The training process takes ap

Table 1: Results on MNIST test set

Model	Accuracy
Our Simple NN	97.5%
CNN (for reference)	99.2%

The learning curve shows that the model converges after about 8 epochs, with minimal improvement there

5. Conclusion

We have demonstrated that a simple feedforward neural network can achieve competitive performance on

Future work could explore the trade-off between model complexity and performance, as well as techniques

References

[1] LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document re

[2] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6